



Documentation

The complete guide to embedding native-speed scripting in your application.

Overview

Ganymede is an embeddable native scripting engine that compiles source code to **real x64 machine code** via JIT. There is no interpreter, no bytecode VM, no garbage collector. The compiled functions are bare native pointers — cast them to your function type and call them with zero overhead, exactly like calling a statically compiled C function. The entire engine ships as a single `Ganymede.dll` with a flat C-style API. Two consumer files are all you need:

File	Language	Description
<code>Ganymede.pas</code>	Delphi / Free Pascal	Dynamic import unit — opaque handles, flat calls, no engine internals
<code>Ganymede.h</code>	C / C++	Single-header dynamic loader — <code>#define GANYMEDE_IMPLEMENTATION</code> in one TU

No packages, no frameworks, no build system integration. Copy the DLL and one file into your project and go.

 Quick Start Delphi

```
uses
  Ganymede;

var
  LEngine: TGnyEngine;
  LResult: TGnyValue;
begin
  if not gny_load(PAnsiChar(CDllPath)) then Exit;
  try
    LEngine := gny_create();
    try
      gny_load_from_string(LEngine,
        PAnsiChar(
          'module mem demo;'#10 +
          'public routine add(a: int64; b: int64): int64;'#10 +
          'begin'#10 +
          '  return a + b;'#10 +
          'end;'#10 +
          'end.'),
        PAnsiChar('demo.gny'));

      if gny_compile(LEngine) then
        begin
          gny_arg_push_int64(LEngine, 30);
          gny_arg_push_int64(LEngine, 12);
          LResult := gny_invoke(LEngine, PAnsiChar('add'), GNY_VT_INT64);
          WriteLn('add(30, 12) = ', LResult.AsInt64); // 42
        end;
      finally
        gny_destroy(LEngine);
      end;
    finally
      gny_unload();
    end;
  end;
```



c

```
#define GANYMEDE_IMPLEMENTATION
#include "Ganymede.h"
#include <stdio.h>

int main(void) {
    if (!gny_load("Ganymede.dll")) return 1;

    GnyEngine engine = gny_create();
    gny_load_from_string(engine,
        "module mem demo;\n"
        "public routine add(a: int64; b: int64): int64;\n"
        "begin\n"
        "    return a + b;\n"
        "end;\n"
        "end.",
        "demo.gny");

    if (gny_compile(engine)) {
        gny_arg_push_int64(engine, 30);
        gny_arg_push_int64(engine, 12);
        GnyValue result = gny_invoke(engine, "add", GNY_VT_INT64);
        printf("add(30, 12) = %lld\n", (long long)result.as_int64);
    }

    gny_destroy(engine);
    gny_unload();
    return 0;
}
```

The Lifecycle

Every integration follows five steps: `gny_load` (load the DLL once) → `gny_create` (new engine instance) → `gny_load_from_string` / `gny_load_from_file` + `gny_compile` (feed source, compile to native) → `gny_arg_push_*` + `gny_invoke` or `gny_get_symbol` (call compiled functions) → `gny_destroy` + `gny_unload` (clean up). Each engine instance is independent — you can run multiple engines concurrently in different threads.

Language Reference

Ganymede scripts use a Pascal-inspired syntax that compiles directly to native x64 machine code. Every script is a **module** — a self-contained compilation unit with types, variables, constants, and routines.

Modules

Every script begins with a module declaration and ends with `end.` — there are three module kinds, each producing a different output:

Kind	Output	Use case
<code>module mem</code>	In-memory JIT code	Call via <code>gny_invoke</code> or <code>gny_get_symbol</code> — the most common mode
<code>module lib</code>	<code>.lib</code> static library file	Reusable library modules imported by other scripts
<code>module dll</code>	<code>.dll</code> dynamic library	Standalone DLL with C-linkage exports for external consumers

text

```
module mem fibonacci;

public routine fib(n: int64): int64;
begin
  if n <= 1 then
    return n;
  end;
  return fib(n - 1) + fib(n - 2);
end;

end.
```

Library modules compile to `.lib` files that other modules can `import`. DLL modules produce standalone `.dll` files with exported symbols.

text

```
module lib test_lib_mathlib;

public routine lib_add(a: int64; b: int64): int64;
begin
  return a + b;
end;

end.
```

text

```
module dll test_dll_mathlib;

public routine lib_add(a: int64; b: int64): int64;
begin
  return a + b;
end;

end.
```

Types

Ganymede is statically typed — every variable, parameter, and return value has a known type at compile time. The type system spans scalars, managed strings, composite types, and pointers.

Scalar Types

Type	Size	Range / Description
int8	1B	Signed -128 to 127
int16	2B	Signed $-32,768$ to $32,767$
int32	4B	Signed -2^{31} to $2^{31}-1$
int64	8B	Signed -2^{63} to $2^{63}-1$
uint8	1B	Unsigned 0 to 255
uint16	2B	Unsigned 0 to $65,535$
uint32	4B	Unsigned 0 to $2^{32}-1$
uint64	8B	Unsigned 0 to $2^{64}-1$
float32	4B	IEEE 754 single-precision (suffix <code>f</code> for literals: <code>3.14f</code>)
float64	8B	IEEE 754 double-precision (default for float literals)
boolean	1B	<code>true</code> or <code>false</code>
char	1B	ANSI character (string literals: <code>"A"</code>)

Type	Size	Range / Description
wchar	2B	Wide character (wide literals: w"X")
pointer	8B	Untyped pointer

 String Types

Type	Encoding	Prefix	Description
string	UTF-8	none	Managed string with automatic reference counting
wstring	UTF-16	w	Wide managed string with automatic reference counting

Both types are reference-counted — assignment, concatenation (+ , +=), and comparison (= , <>) just work. wstring uses UTF-16 encoding internally (2 bytes per code unit) and is prefixed with w : w"Hello" . No manual memory management needed for either type.

```
var
  s: string = "Hello UTF-8";
  ws: wstring = w"Hello UTF-16";
```

text

 Record Types

Records group fields into structured types with support for packed layout, explicit alignment, inheritance, nested records, and record literals. Fields are accessed with dot notation and records can contain any type including other records and managed strings.

```
type
  TPoint = record
    x: int32;
    y: int32;
  end;

public routine pointsum(): int32;
var
  p: TPoint;
begin
  p.x := 10;
  p.y := 20;
  return p.x + p.y;
end;
```

text

Record literals initialize all fields in one expression:

```
public routine pointlit(): int32;
var
  p: TPoint;
begin
  p := TPoint(x: 100, y: 200);
  return p.x + p.y;  // 300
end;
```

text

 Packed records eliminate all padding between fields — useful for binary file formats, network protocols, and hardware register maps:

```
type
  TPackedHeader = record packed
    tag: uint8;
    flags: uint8;
    length: uint16;
    payload: uint32;
  end;
```

text

 **Aligned records** force a specific memory alignment boundary — critical for SIMD, cache-line alignment, and DMA buffers:

```
type
  TAligned16 = record align(16)
    x: float32;
    y: float32;
    z: float32;
    w: float32;
  end;
```

text

You can combine both — `record packed align(8)` gives packed layout with 8-byte alignment:

```
type
  TCacheLinePacked = record packed align(8)
    counter: uint64;
    flags: uint32;
    tag: uint8;
  end;
```

text

 **Inheritance** — a derived record includes all base fields:

```
type
  TBase = record
    id: int64;
  end;

type
  TDerived = record(TBase)
    value: int32;
    ratio: float64;
  end;

public routine inheritance(): int64;
var
  d: TDerived;
begin
  d.id := 100;
  d.value := 200;
  d.ratio := 1.5;
  return d.id + d.value;  // 300
end;
```

text

 **Nested records** support deep field access:

```
type
  TLine = record
    start: TPoint;
    finish: TPoint;
  end;

public routine nestedrec(): int32;
var
  ln: TLine;
begin
  ln.start.x := 1;
  ln.start.y := 2;
  ln.finish.x := 10;
  ln.finish.y := 20;
  return ln.start.x + ln.start.y + ln.finish.x + ln.finish.y;  // 33
end;
```

text

Array Types

Static arrays have a fixed size known at compile time. **Dynamic arrays** are heap-allocated and resizable via `setlength`. Use `len()` to query length at runtime. Both support indexing with `[]`.

✦ Static arrays:

```
type
  TIntArray = array[0..4] of int32;

public routine intarr(): int32;
var
  a: TIntArray;
begin
  a[0] := 10;
  a[4] := 50;
  return a[0] + a[4];  // 60
end;
```

Inline array types don't need a named type: `var a: array[0..2] of int32;`

🔄 **Dynamic arrays** — heap-allocated, resizable at runtime via `setlength`, queried with `len()`:

```
public routine dynarr_basic(): int32;
var
  a: array of int32;
  s: int32;
begin
  setlength(a, 5);
  a[0] := 10;
  a[1] := 20;
  a[2] := 30;
  a[3] := 40;
  a[4] := 50;
  s := a[0] + a[1] + a[2] + a[3] + a[4];
  return s;  // 150
end;
```

`len()` returns the current length — `0` for unallocated arrays:

```
public routine dynarr_len(): int64;
var
  a: array of int32;
begin
  setlength(a, 10);
  return len(a);  // 10
end;
```

Loop fill and sum with dynamic arrays — identical indexing syntax to static:

```
public routine dynarr_loopsum(): int32;
var
  a: array of int32;
  i: int32;
  s: int32;
begin
  setlength(a, 10);
  for i := 0 to 9 do
    a[i] := (i + 1) * 10;
  end;

  s := 0;
  for i := 0 to 9 do
    s := s + a[i];
  end;

  return s;  // 550
end;
```

text

text

text

text

👉 Pointer Types

Typed pointers reference a specific type. Use `address of` (or the shorthand `&`) to take an address, `^` to dereference, and `nil` for null pointers. Named pointer types are also supported: `type PInt32 = pointer to int32;`

```
public routine ptr_i32(): int32;
var
  x: int32;
  p: pointer to int32;
begin
  x := 42;
  p := address of x;
  return p^;  // 42
end;
```

text

The `&` shorthand does the same thing:

```
public routine ptr_ampersand(): int32;
var
  x: int32;
  p: pointer to int32;
begin
  x := 77;
  p := &x;
  return p^;  // 77
end;
```

text

Write through pointers with `p^ := 99;`, compare with `nil`, and pass pointers to routines for by-reference semantics:

```
routine set_value(p: pointer to int32; val: int32);
begin
  p^ := val;
end;

public routine ptr_passref(): int32;
var
  x: int32;
begin
  x := 0;
  set_value(address of x, 123);
  return x;  // 123
end;
```

text

🔄 Overlay Types (Unions)

Overlays share memory between fields — like C unions. The total size equals the largest field. Write to one field, read from another for type-punning (e.g., reinterpreting an `int32` as `float32` via IEEE 754 bit patterns).

```
type
  TIntFloat = overlay
    asInt: int32;
    asFloat: float32;
end;
```

text

🏷️ **Tagged unions** embed an anonymous overlay inside a record, giving you a discriminator field alongside the union:


```

type
  TTaggedValue = record
    tag: int32;
  overlay
    intVal: int32;
    floatVal: float32;
  end;
end;

```

🧩 **Overlays with anonymous records** create C-style unions with struct arms — each arm shares the same memory but has its own named fields:

```

type
  TEvent = overlay
    record
      x: int32;
      y: int32;
    end;
    record
      key: int32;
      modifiers: int32;
    end;
end;

```

🎨 Choices (Enums) & Sets

Choices are enumerated types with auto-assigned or explicit ordinal values. **Sets** are bitfield types supporting the `in` operator for membership testing with values, comma lists, and ranges.

```

type
  TColor = choices(red, green, blue);
  TSeverity = choices(none = 0, warn = 5, fail = 10);
  TSmallSet = set of 0..63;

public routine setmember(): int32;
var
  s: TSmallSet;
begin
  s := [1, 3, 5..10];
  if 7 in s then
    return 1;
  end;
  return 0;
end;

```

⚙️ Routine Types (Function Pointers)

First-class function pointers that can be assigned, passed as callback parameters, stored in arrays, and called indirectly. Define a routine type, take the address of a compatible function, and call through the pointer.

```

type
  TBinOp = routine(int32, int32): int32;

routine add(a: int32; b: int32): int32;
begin
  return a + b;
end;

routine apply(f: TBinOp; x: int32; y: int32): int32;
begin
  return f(x, y);
end;

public routine test_callback(): int32;
begin
  return apply(address of add, 6, 7);  // 13
end;

```

Variables & Constants

Variables are declared in `var` blocks with explicit types and optional initializers. **Constants** are declared in `const` blocks and cannot be reassigned — the compiler enforces immutability. **Module-level variables** (declared outside any routine) act as globals visible to all routines in the module.

```

public routine compute(x: int32): int32;
const
  BASE: int32 = 100;
  MULT: int32 = 3;
var
  result: int32;
begin
  result := BASE + x * MULT;
  return result;
end;

```

Operators

Arithmetic: `+`, `-`, `*`, `/` (float division, always returns float), `div` (integer division), `mod` (integer modulus). **Bitwise:** `and`, `or`, `xor`, `not`, `shl`, `shr`. **Comparison:** `=`, `<>`, `<`, `<=`, `>`, `>=`. **Assignment:** `:=`, `+=`, `-=`, `*=`, `/=`. **Other:** `in` (set membership), `address of` / `&` (take address), `^` (dereference).

Control Flow

If/then/end — conditional execution. **While/do/end** — pre-condition loop. **For/to/end** and **for/downto/end** — counted loops with automatic iterator. **Repeat/until** — post-condition loop. **Leave** breaks out of the current loop. **Skip** jumps to the next iteration.

```
// While loop
while i <= n do
  s := s + i;
  i := i + 1;
end;

// For loop (to and downto)
for i := 1 to n do
  s := s + i;
end;

// Repeat/until
repeat
  s := s + i;
  i := i + 1;
until i > n;

// Leave (break) and skip (continue)
while i <= n do
  if s > 10 then
    leave;
  end;
  rem := i mod 2;
  if rem = 0 then
    skip;
  end;
  s := s + i;
  i := i + 1;
end;
```

Match Statement

Pattern matching on integer values with support for comma-separated labels, ranges, and an optional `else` branch. All match labels must be compile-time constants.

```
match x of
  0:      return 100;
  1..3:   return 200;
  4, 7, 9: return 300;
  10..15: return 400;
else
  return -1;
end;
```

Routines

Routines are the basic unit of executable code. Mark them `public` to export from the module (visible to `gny_invoke` and `gny_get_symbol`). Private routines (no `public` keyword) are internal to the module.

 **Linkage:** By default, all routines use **C linkage** (`cdecl` calling convention). This means `gny_get_symbol` returns a pointer you can cast to a C-compatible function type and call directly.

For **overloaded routines** (multiple routines with the same name but different parameter signatures), you must use the `cpplink` keyword. This tells the compiler to use C++ name mangling so each overload gets a unique export name. Place `cpplink` between `routine` and the function name:

```
// C linkage (default) – unique name, callable via gny_get_symbol
public routine add(a: int64; b: int64): int64;
begin
  return a + b;
end;

// C++ linkage – overloaded names require cpplink keyword
public routine cpplink compute(a: int64): int64;
begin
  return a * 10;
end;

public routine cpplink compute(a: int64; b: int64): int64;
begin
  return a + b;
end;
```

Call overloaded `cpplink` routines from the host through `gny_invoke` — push the right argument types and the engine resolves the correct overload automatically. For importing `cpplink` routines from external DLLs, use the same keyword:

```
routine cpplink compute(a: int64): int64;
  external "my_overloaded.dll";

routine cpplink compute(a: int64; b: int64): int64;
  external "my_overloaded.dll";
```

🔔 **Variadic arguments** — routines that accept a variable number of arguments using `...`. Inside the body, `varargs.next(Type)` consumes the next argument and `varargs.count` returns how many variadic arguments were passed.

```
public routine sum_ints(count: int32; ...): int64;
var
  result: int64 = 0;
  i: int32;
begin
  for i := 0 to count - 1 do
    result += varargs.next(int64);
  end
  return result;
end;
```

📦 Imports & Externals

🔗 Importing Modules

The `import` clause pulls in other Ganymede modules by name. The compiler searches directories registered via `gny_add_lib_path` on the host side. Imported symbols are accessed with qualified names: `module_name.symbol_name`.

```
module mem test_mem_import_basic;
import test_lib_mathlib;

public routine main(): int64;
begin
  return test_lib_mathlib.lib_add(10, 20);  // 30
end;

end.
```

🌐 External Declarations (FFI)

Ganymede can link directly against **native DLLs** and **static `.lib` files** produced by any Win64 C or C++ compiler (MSVC, Clang, MinGW, etc.). The compiler resolves symbols at link time — no wrapper code, no glue layer.

 **DLL externals** — use the DLL name (with or without `.dll` extension):

```
module mem test_mem_external_dll;

public routine abs(x: int64): int64;
  external "msvcrt";

public routine main(): int64;
begin
  return abs(-42);  // 42
end;

end.
```

text

 **Static `.lib` externals** — use the filename with `.lib` extension. This links against standard Win64 COFF `.lib` files with C linkage:

```
module mem example;

// Link against a .lib compiled by any Win64 C compiler
public routine lib_add(a: int64; b: int64): int64;
  external "mathlib.lib";

public routine main(): int64;
begin
  return lib_add(10, 20);  // 30
end;

end.
```

text

 **C++ linkage from `.lib`** — use `cpplink` to import overloaded functions with Itanium C++ name mangling:

```
module mem example;

// C++ mangled symbols from a .lib
routine cpplink compute(a: int64): int64;
  external "mathlib.lib";

routine cpplink compute(a: int64; b: int64): int64;
  external "mathlib.lib";

// C linkage symbol from the same .lib
routine add(a: int64; b: int64): int64;
  external "mathlib.lib";

public routine main(): int64;
begin
  return compute(5) + compute(3, 4) + add(10, 20);
end;

end.
```

text

The engine searches lib paths registered via `gny_add_lib_path` on the host side.

Host Function Binding

Import host-side (Delphi/C) functions into the script at runtime via `gny_import_host`. See the  [Host Interop](#) section below for the complete guide.

Conditional Compilation

Preprocessor directives control which code gets compiled. Ganymede predefines platform, build, and optimization symbols automatically. Use `@define` / `@undef` to create your own, and `#ifdef` / `#ifndef` / `#elseif` / `#else` / `#endif` to conditionally include or exclude code blocks. You can also define symbols from the host side with `gny_set_define` before compilation.

```
@define MY_FEATURE

#ifdef WINDOWS
    // Windows-specific code
#elif LINUX
    // Linux-specific code
#else
    // Fallback
#endif
```

Predefined symbols: `GANYMEDE` (always), `WINDOWS` , `WIN64` , `MSWINDOWS` , `CPUX64` , `TARGET_WIN64` (platform), `BUILD_MEM` / `BUILD_LIB` / `BUILD_DLL` (module kind), `APPTYPE_CONSOLE` (host app type), `DEBUG` (opt=none), `RELEASE` (opt>none).

Built-in Intrinsics

Intrinsic	Description
<code>size(Type)</code>	 Size in bytes of any type at compile time — scalars, records, arrays
<code>len(x)</code>	 Runtime length of a dynamic array or managed string
<code>utf8(ws)</code>	 Convert a <code>wstring</code> to a UTF-8 encoded <code>pointer</code>
<code>write(fmt, ...)</code>	 Output to console with printf-style format string
<code>writeln(fmt, ...)</code>	 Output to console with newline
<code>getmem(p)</code>	 Allocate heap memory for a typed pointer (size inferred from type)
<code>freemem(p)</code>	 Free heap-allocated memory
<code>resizemem(p, size)</code>	 Resize a heap allocation (data survives)
<code>setlength(arr, n)</code>	 Allocate or resize a dynamic array

```
public routine test_getmem_i32(): int32;
var
    p: pointer to int32;
    result: int32;
begin
    getmem(p);
    p^ := 42;
    result := p^;
    freemem(p);
    return result; // 42
end;
```

```
public routine test_size_in_expr(): int32;
var
    s: int64;
begin
    s := size(int32) + size(int64); // 4 + 8 = 12
    return int32(s);
end;
```


Managed Strings

`string` and `wstring` types are reference-counted and managed automatically. Assignment copies the reference (with copy-on-write semantics), concatenation creates a new string, and comparison works with `=` and `<>`. You never need to free a managed string — the runtime handles cleanup when variables go out of scope. `wstring` is UTF-16 encoded internally.

text

```
var
  s1: string = "Hello";
  s2: string = " World";
  s3: string;
begin
  s3 := s1 + s2;           // "Hello World"
  s1 += "!";               // "Hello!"
  if s3 = "Hello World" then
    // ...
  end;
```

API Reference

Every function below is available through `Ganymede.pas` (Delphi/FPC) and `Ganymede.h` (C/C++) after calling `gny_load`. All string parameters are null-terminated UTF-8 (`PAnsiChar` / `const char*`).

Loader

Function	Description
<code>gny_load(path) → Boolean</code>	 Load <code>Ganymede.dll</code> from the given path. Returns <code>True</code> on success. Safe to call if already loaded.
<code>gny_unload()</code>	 Unload the DLL and reset all function pointers to nil. Destroy all engines first.
<code>gny_is_loaded() → Boolean</code>	 Check whether the DLL is currently loaded.

Lifecycle

Function	Description
<code>gny_create() → TGnyEngine</code>	 Create a new engine instance. Each instance is fully independent.
<code>gny_destroy(engine)</code>	 Destroy an engine and free all resources (JIT code, symbol tables, errors).
<code>gny_version() → PAnsiChar</code>	 Engine version string. Owned by the DLL — do not free.
<code>gny_free(ptr)</code>	 Free a heap-allocated string returned by <code>gny_get_errors</code> , <code>gny_get_symbol_names</code> , or <code>gny_get_ssa_dump</code> .

Source Loading

Function	Description
<code>gny_load_from_string(engine, source, filename)</code>	 Load source from a UTF-8 string. The filename appears in error messages.
<code>gny_load_from_file(engine, filename)</code>	 Load source from a file on disk.

 Compilation

Function	Description
<code>gny_compile(engine)</code> → <code>Boolean</code>	 Compile loaded source to native x64 code. Returns <code>True</code> on success. You can recompile on the same engine — load new source and call compile again. Host imports survive recompilation.

 Argument Building & Invocation

Push-based calling: push arguments one by one, then invoke. Arguments are **auto-cleared** after each invoke.

Function	Description
<code>gny_arg_push_int8(engine, value)</code>	Push <code>int8</code>
<code>gny_arg_push_int16(engine, value)</code>	Push <code>int16</code>
<code>gny_arg_push_int32(engine, value)</code>	Push <code>int32</code>
<code>gny_arg_push_int64(engine, value)</code>	Push <code>int64</code>
<code>gny_arg_push_uint8(engine, value)</code>	Push <code>uint8</code>
<code>gny_arg_push_uint16(engine, value)</code>	Push <code>uint16</code>
<code>gny_arg_push_uint32(engine, value)</code>	Push <code>uint32</code>
<code>gny_arg_push_uint64(engine, value)</code>	Push <code>uint64</code>
<code>gny_arg_push_float32(engine, value)</code>	Push <code>float32</code>
<code>gny_arg_push_float64(engine, value)</code>	Push <code>float64</code>
<code>gny_arg_push_pointer(engine, value)</code>	Push <code>pointer</code>
<code>gny_arg_clear(engine)</code>	 Manually clear the arg stack (rarely needed).
<code>gny_invoke(engine, name, returnType)</code> → <code>TGnyValue</code>	 Call a compiled function by name. <code>returnType</code> is a <code>GNV_VT_*</code> constant. Returns result as a tagged value.

Return type constants: `GNV_VT_VOID` (0), `GNV_VT_INT8` (1), `GNV_VT_INT16` (2), `GNV_VT_INT32` (3), `GNV_VT_INT64` (4), `GNV_VT_UINT8` (5), `GNV_VT_UINT16` (6), `GNV_VT_UINT32` (7), `GNV_VT_UINT64` (8), `GNV_VT_FLOAT32` (9), `GNV_VT_FLOAT64` (10), `GNV_VT_POINTER` (11).

 Direct Function Pointers

For maximum performance, bypass `gny_invoke` entirely. Get a raw function pointer with `gny_get_symbol`, cast it to your native function type, and call it directly — identical performance to calling a statically compiled function.

```
type
  TFibFunc = function(n: Int64): Int64;
var
  LFib: TFibFunc;
begin
  LFib := gny_get_symbol(LEngine, PAnsiChar('fib'));
  WriteLn(LFib(30)); // 832040 – zero overhead, pure native call
end;
```


 Symbols

Function	Description
<code>gny_get_symbol(engine, name) → Pointer</code>	 Get the native function pointer for a <code>public</code> symbol. Cast and call directly.
<code>gny_has_symbol(engine, name) → Boolean</code>	 Check whether a symbol exists.
<code>gny_get_symbol_names(engine) → PAnsiChar</code>	 JSON array of all exported symbol names. Free with <code>gny_free</code> .

 Configuration

Function	Description
<code>gny_set_optimization_level(engine, level)</code>	 Set optimization: <code>GNY_OPT_NONE</code> (0), <code>GNY_OPT_BASIC</code> (1), <code>GNY_OPT_FULL</code> (2).
<code>gny_get_optimization_level(engine) → Integer</code>	 Query current optimization level.
<code>gny_set_output_path(engine, path)</code>	 Output directory for <code>module dll</code> / <code>module lib</code> compilation.
<code>gny_add_lib_path(engine, path)</code>	 Add a search path for <code>import</code> resolution and <code>.lib</code> lookup. Callable multiple times.
<code>gny_set_dump_ir(engine, value)</code>	 Enable/disable SSA IR dump generation during compilation.
<code>gny_get_ssa_dump(engine) → PAnsiChar</code>	 Retrieve SSA IR dump. Free with <code>gny_free</code> .

 Conditional Compilation (Host-Side)

Function	Description
<code>gny_set_define(engine, name, value)</code>	 Define a symbol visible to <code>@ifdef</code> / <code>@ifndef</code> . Empty string for valueless defines.
<code>gny_undefine(engine, name)</code>	 Remove a previously defined symbol.
<code>gny_is_defined(engine, name) → Boolean</code>	 Check whether a symbol is defined.

 Error Reporting

Function	Description
<code>gny_print_errors(engine)</code>	 Print all errors to console with color-coded severity.
<code>gny_get_errors(engine) → PAnsiChar</code>	 Errors as JSON array (<code>severity</code> , <code>code</code> , <code>message</code> , <code>location</code>). Free with <code>gny_free</code> .
<code>gny_has_errors(engine) → Boolean</code>	 Check whether any errors exist.

 Status Callback

Function	Description
<code>gny_set_status_callback(engine, callback, userData)</code>	 Register a callback for compilation status messages. Pass <code>nil</code> / <code>NULL</code> to unregister.

```
Delphi: TGnyApiStatusHandler = procedure(const AText: PAnsiChar; const AUserData: Pointer); cdecl;
C: typedef void (*GnyStatusHandler)(const char* text, void* user_data);
```

The `text` pointer is stack-local — copy it if you need it beyond the callback invocation.

 Debug

Function	Description
<code>gny_report_leaks(engine)</code>	 Print unfreed heap allocations from compiled script code. Call before <code>gny_destroy</code> .

 Host Interop

`gny_import_host` injects native functions from your application into the scripting environment. The script calls them as if they were built-in routines. Register host functions **before** calling `gny_compile` — they persist across recompilations on the same engine.

Signature: `gny_import_host(engine, name, addr, paramTypes, paramCount, returnType, linkage)`

Parameter	Type	Description
<code>engine</code>	<code>TGnyEngine</code>	Engine instance
<code>name</code>	<code>PAnsiChar</code>	Function name as seen by the script
<code>addr</code>	<code>Pointer</code>	Address of the host function
<code>paramTypes</code>	<code>PInteger</code>	Pointer to array of <code>GNV_VT_*</code> constants. <code>nil</code> if no params.
<code>paramCount</code>	<code>Integer</code>	Number of parameters (0 if none)
<code>returnType</code>	<code>Integer</code>	<code>GNV_VT_*</code> constant for the return, or <code>GNV_VT_VOID</code>
<code>linkage</code>	<code>Integer</code>	<code>GNV_LINK_DEFAULT</code> (0) or <code>GNV_LINK_C</code> (1)

◆ Example — Function with parameters and return value

```
function host_add(const A, B: Int64): Int64;
begin
    Result := A + B;
end;

const
    CParamsInt64x2: array[0..1] of Integer = (GNV_VT_INT64, GNV_VT_INT64);

begin
    gny_import_host(LEngine,
        PAnsiChar('host_add'),           // name in script
        @host_add,                       // function address
        @CParamsInt64x2[0], 2,           // param types array + count
        GNV_VT_INT64,                    // return type
        GNV_LINK_DEFAULT);               // linkage
```

◆ Example — Void function with no parameters

```
procedure host_increment_counter();
begin
  Inc(GHostCallCount);
end;

begin
  gny_import_host(LEngine,
    PAnsiChar('host_increment_counter'),
    @host_increment_counter,
    nil, 0,                                // no params: nil + 0
    GNY_VT_VOID,                            // no return value
    GNY_LINK_DEFAULT);
```

The script can then call `host_add(10, 20)` or `host_increment_counter()` directly — the JIT compiler generates a native `call` instruction to your function pointer.

🌐 C Interop

Ganymede is designed for seamless interop with C and any language that supports `cdecl` calling conventions. The entire boundary — both into and out of the scripting engine — uses **C-compatible** conventions.

🔄 Calling Convention

All Ganymede routines use `cdecl` by default. This means:

- `gny_get_symbol` returns a C-callable function pointer. Cast it to your native function type and call it with zero overhead — no marshalling, no boxing, no lookup cost.
- `gny_import_host` accepts any `cdecl` function pointer. The JIT compiler generates a direct native `call` instruction to your function address.
- `external "dllname"` calls functions from arbitrary native DLLs using `cdecl` — the standard C calling convention on x64 Windows.

🔧 From C / C++

```
// Get a compiled function pointer and call it directly
typedef int64_t (*FnAdd)(int64_t, int64_t);
FnAdd add_fn = (FnAdd)gny_get_symbol(engine, "add");
int64_t result = add_fn(30, 12); // 42 — pure native call
```

🔧 From Delphi

```
type
  TAddFunc = function(a, b: Int64): Int64; cdecl;
var
  LAdd: TAddFunc;
begin
  LAdd := gny_get_symbol(LEngine, PAnsiChar('add'));
  WriteLn(LAdd(30, 12)); // 42
end;
```

🔗 Calling Native Code from Script

Scripts can call into native DLLs and static `.lib` files directly — the compiler handles symbol resolution at link time:

📦 **DLL imports** — any system or third-party DLL:

// Call C runtime abs() from msvcrt.dll
public routine abs(x: int64): int64;
 external "msvcrt";

text

 **Static .lib imports** — standard Win64 COFF .lib files from any C compiler (MSVC, Clang, MinGW, etc.):

// Link against a C-compiled .lib with C linkage
public routine fast_hash(data: pointer; len: int64): uint64;
 external "myhash.lib";

text

 **C++ .lib imports** — use `cpplink` for overloaded functions with Itanium C++ name mangling:

// C++ mangled overloads from a .lib
routine cpplink compute(a: int64): int64;
 external "mathlib.lib";

routine cpplink compute(a: int64; b: int64): int64;
 external "mathlib.lib";

text

Overloaded Exports

For functions with the same name but different parameter signatures, use `cpplink` to enable C++ name mangling. This lets multiple overloads coexist as distinct exports. Call them from the host via `gny_invoke` — the engine resolves the correct overload based on pushed argument types.

Memory & String Contract

 **Encoding:** All strings crossing the DLL boundary are **null-terminated UTF-8** (`PAnsiChar` in Delphi, `const char*` in C). Inside scripts, `string` is UTF-8 and `wstring` is UTF-16.

 **Ownership:**

Scenario	Who frees?
Strings you pass to the API	You own them — the engine copies what it needs
Strings from <code>gny_get_errors</code> , <code>gny_get_symbol_names</code> , <code>gny_get_ssa_dump</code>	You must free with <code>gny_free</code>
String from <code>gny_version</code>	Engine-owned — do not free
Strings passed to status callbacks	Stack-local — copy if needed

 **Thread safety:** Each engine handle is fully independent with no shared state. Multiple engines can run concurrently in different threads. A single engine must not be accessed from multiple threads simultaneously.

Building from Source

	Requirement
Host OS	Windows 10/11 x64
Compiler	Delphi 12.x or higher

Open the project group in `src\`, build the DLL project — output goes to `lib\bin\Ganymede.dll`. Copy the DLL plus `lib\pascal\Ganymede.pas` (or `lib\c\include\Ganymede.h`) to your consumer project and you're ready to go. 

BNF Grammar

This grammar covers all currently implemented language features. Productions use `|` for alternatives, `[ ]` for optional elements, `{ }` for zero-or-more repetition, and `( )` for grouping.

Module Structure

module	= module-header { declaration } [module-body] "end" "." .	text
module-header	= "module" module-kind identifier ";" [import-clause] .	
module-kind	= "mem" "lib" "dll" .	
import-clause	= "import" identifier { "," identifier } ";" .	
module-body	= "begin" statement-list .	

Declarations

declaration	= type-decl var-decl const-decl routine-decl .	text
type-decl	= "type" identifier "=" type-def ";" .	
var-decl	= "var" identifier ":" type-ref ["=" expression] ";" .	
const-decl	= "const" identifier ":" type-ref "=" expression ";" .	

Routine Declarations

routine-decl	= ["public"] "routine" ["cpplink"] identifier "(" [param-list] ")" [":" type-ref] ";" (routine-body external-decl) .	text
param-list	= param { ";" param } [";" "..."] "..." .	
param	= identifier ":" type-ref .	
routine-body	= { const-decl var-decl } "begin" statement-list "end" ";" .	
external-decl	= "external" string-literal ";" . (* string is DLL name: "msvcrt", "kernel32.dll" or static lib: "mylib.lib" *)	

 Type Definitions

text

type-def	= record-def array-def overlay-def choices-def set-def routine-def pointer-def .
record-def	= "record" ["packed"] ["align" "(" integer ")"] ["(" identifier ")"] { field-decl anon-overlay } "end" .
field-decl	= identifier ":" type-ref ";" .
anon-overlay	= "overlay" { field-decl anon-record } "end" ";" .
anon-record	= "record" { field-decl } "end" ";" .
array-def	= "array" "[" expression ".." expression "]" "of" type-ref "array" "of" type-ref .
overlay-def	= "overlay" { field-decl anon-record } "end" .
choices-def	= "choices" "(" enum-member { "," enum-member } ")" .
enum-member	= identifier ["=" integer] .
set-def	= "set" "of" expression ".." expression "set" "of" identifier .
routine-def	= "routine" "(" [type-list] ")" [":" type-ref] .
type-list	= type-ref { "," type-ref } .
pointer-def	= "pointer" "to" type-ref .

 Type References

text

type-ref	= "int8" "int16" "int32" "int64" "uint8" "uint16" "uint32" "uint64" "float32" "float64" "boolean" "char" "wchar" "string" "wstring" "pointer" ["to" type-ref] "array" "[" expression ".." expression "]" "of" type-ref "array" "of" type-ref identifier .
----------	---

Statements

text

```

statement-list = { statement ";" } .

statement      = assign-stmt | call-stmt | if-stmt | while-stmt
                | for-stmt | repeat-stmt | match-stmt | return-stmt
                | leave-stmt | skip-stmt | write-stmt | mem-stmt
                | setlength-stmt .

assign-stmt    = designator assign-op expression .
assign-op      = "[:=" | "+=" | "-=" | "*=" | "/=" .
call-stmt      = designator "(" [ arg-list ] ")" .

if-stmt        = "if" expression "then" statement-list
                { "else" "if" expression "then" statement-list }
                [ "else" statement-list ]
                "end" .

while-stmt     = "while" expression "do" statement-list "end" .
for-stmt       = "for" identifier "[:=" expression ( "to" | "downto" )
                expression "do" statement-list "end" .
repeat-stmt    = "repeat" statement-list "until" expression .

match-stmt     = "match" expression "of"
                { match-arm }
                [ "else" statement-list ]
                "end" .
match-arm      = match-label { "," match-label } ":" statement-list .
match-label    = expression [ ".." expression ] .

return-stmt    = "return" [ expression ] .
leave-stmt     = "leave" .
skip-stmt      = "skip" .

write-stmt     = ( "write" | "writeln" ) "(" arg-list ")" .
mem-stmt       = ( "getmem" | "freemem" ) "(" designator ")"
                | "resizemem" "(" designator "," expression ")" .
setlength-stmt = "setlength" "(" designator "," expression ")" .

```

+ Expressions

text

```
expression      = unary-expr { binary-op unary-expr } .
unary-expr      = [ unary-op ] primary .
unary-op        = "-" | "not" | "address" "of" | "&" .

primary         = integer-literal | float-literal | string-literal
                  | wide-string-literal | "true" | "false" | "nil"
                  | designator [ "(" [ arg-list ] ")" ]
                  | record-literal | set-literal
                  | "(" expression ")" | type-cast
                  | "size" "(" type-ref ")"
                  | "len" "(" expression ")"
                  | "utf8" "(" expression ")"
                  | "varargs" "." ( "next" "(" type-ref ")"
                                      | "count" | "copy" "(" ")" ) .

designator       = identifier { "." identifier | "[" expression "]" | "^" } .
record-literal  = identifier "(" field-init { "," field-init } ")" .
field-init      = identifier ":" expression .
set-literal     = "[" [ set-element { "," set-element } ] "]" .
set-element     = expression [ ".." expression ] .
type-cast       = type-ref "(" expression ")" .
arg-list        = expression { "," expression } .

binary-op       = "+" | "-" | "*" | "/" | "div" | "mod"
                  | "and" | "or" | "xor" | "shl" | "shr"
                  | "=" | "<>" | "<" | "<=" | ">" | ">="
                  | "in" .
```

🔧 Conditional Compilation Directives

text

```
directive       = "@define" identifier
                  | "@undef" identifier
                  | "@ifdef" identifier
                  | "@ifndef" identifier
                  | "@elseif" identifier
                  | "@else"
                  | "@endif" .
```

🔑 Operator Precedence (highest to lowest)

Precedence	Operators	Description
1 (highest)	<code>^</code> <code>.</code> <code>[]</code> <code>()</code>	Dereference, field access, index, call
2	<code>not</code> <code>-</code> (unary) & <code>address of</code>	Unary operators
3	<code>*</code> <code>/</code> <code>div</code> <code>mod</code> <code>and</code> <code>shl</code> <code>shr</code>	Multiplicative
4	<code>+</code> <code>-</code> <code>or</code> <code>xor</code>	Additive
5	<code>=</code> <code><></code> <code><</code> <code><=</code> <code>></code> <code>>=</code> <code>in</code>	Comparison / membership
6 (lowest)	<code>:=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code>	Assignment

 Lexical Elements

text

```
identifier      = letter { letter | digit | "_" } .
integer-literal = digit { digit } | "0x" hex-digit { hex-digit } .
float-literal   = digit { digit } "." digit { digit } [ "f" ] .
string-literal  = '"' { character } '"' .
wide-str-lit    = 'w"' { character } '"' .
line-comment    = "//" { character } newline .
block-comment   = "/*" { character } "*/" .
```

Ganymede™ — Embeddable Native Scripting Engine

Copyright © 2026-present tinyBigGAMES™ LLC — All Rights Reserved.